

Technische handleiding & data-recovery

Voor technisch beheer · Intern Studentbeheersysteem (SIS) · IUASR

Dit document is bestemd voor **technisch personeel/beheerders**. Het beschrijft de architectuur, het maken van back-ups en de **herstelprocedure**. Bewaar het vertrouwelijk.

1. Architectuur & stack

- **Applicatie:** PHP + Laravel (server-rendered; geen kale PHP/WordPress).
- **Database:** MySQL/MariaDB (InnoDB, echte foreign keys, surrogaatsleutels).
- **Authenticatie:** Microsoft Entra ID (SSO/OIDC) — nooit een eigen login bouwen. In de ontwikkelomgeving een tijdelijke dev-login.
- **Netwerk:** draait intern (intranet), IP-beperkt, gescheiden van het publieke aanmeldportaal.
- **Gevoelige data:** BSN en rekeningnummer versleuteld (Laravel-encryptie met **APP_KEY**); inzage/mutatie gelogd (audit-log).

2. Omgeving (referentie)

PHP	~/php/8.3/php.exe
Composer	~/bin/composer.phar
Database-server	MariaDB (portable) — ~/mariadb/mariadb-11.4.9-winx64/bin/
DB-host / poort	127.0.0.1 : 3307
Database / gebruiker	iuasr_sis / iuasr_sis
Configuratie	.env (in de projectmap) — bevat DB-gegevens en APP_KEY

APP_KEY is kritiek. Zonder de originele APP_KEY zijn de versleutelde velden (BSN, rekeningnummer) onherstelbaar. De sleutel zit in .env en wordt meegenomen in de back-up.

E-mail (resultaten mailen)

De examencommissie kan definitieve resultaten per e-mail naar studenten sturen. In

ontwikkeling staat `MAIL_MAILER=log`: e-mails worden naar `storage/logs/laravel.log` geschreven, er gaan GEEN echte e-mails uit (AVG, synthetische data). In **productie** configureert u de IUASR-mailserver in `.env`:

```
MAIL_MAILER=smtp
MAIL_HOST=<smtp.iuasr.nl>
MAIL_PORT=587
MAIL_USERNAME=<gebruiker>
MAIL_PASSWORD=<wachtwoord>
MAIL_ENCRYPTION=tls
MAIL_FROM_ADDRESS=noreply@iuasr.nl
MAIL_FROM_NAME="IUASR Studentenzaken"
```

Elke student ontvangt individueel de eigen (ondertekende) cijferlijst als bijlage; verzending wordt gelogd. Overweeg voor grote aantallen een queue (`QUEUE_CONNECTION + worker`).

3. Back-up maken

Een volledige back-up wordt gemaakt via de webapplicatie:

1. Log in als **Beheerder**.
2. Ga naar **Beheer → Back-up & herstel**.
3. Geef een sterk **wachtwoord** op (minimaal 8 tekens) en bevestig.
4. Klik op **Back-up genereren & downloaden**. U ontvangt een met AES-256 versleutelde ZIP.

De ZIP bevat: `database.sql` (volledige dump), de applicatiebroncode en webpagina's, `.env` (incl. `APP_KEY`) en de geüploade bestanden (`storage/app`). Niet inbegrepen: `vendor/`, `.git/` en de referentiemap `IUASR/`. Het wachtwoord wordt **nergens opgeslagen**; downloaden wordt ge-audit-logd.

Advies: bewaar back-ups versleuteld op een beveiligde, interne locatie en hanteer een bewaarschema (bijv. wekelijks + vóór elke update). Overweeg een periodieke, geautomatiseerde back-up naar een netwerkschijf.

4. Herstelprocedure (recovery)

Terugzetten is een bewuste beheerhandeling en gebeurt **niet** vanuit de draaiende applicatie (die kan zichzelf niet veilig overschrijven en een database-restore wist bestaande data). Voer de

stappen uit op de server.

Stap 1 — Archief uitpakken

De Windows Verkenner kan een AES-versleutelde ZIP **niet** openen. Gebruik het meegeleverde commando (of 7-Zip/WinRAR):

```
~/php/8.3/php.exe artisan backup:uitpakken "pad/naar/iuasr-sis-backup-JJJJMMDD-UUMM.zip" --doel="pad/naar/hersteld"
```

U wordt om het wachtwoord gevraagd (of geef --wachtwoord=...). Het commando verifieert het wachtwoord en pakt uit naar de doelmap.

Stap 2 — Bestanden plaatsen

Plaats de uitgepakte bestanden in de webroot/projectmap van de (interne) server. Zet ook storage/app (geüploade documenten) terug.

Stap 3 — Afhankelijkheden herstellen

```
~/bin/composer.phar install --no-dev --optimize-autoloader
```

Stap 4 — Database terugzetten

Maak (indien nodig) een lege database en importeer de dump. De dump bevat zowel het schema als de data (DROP/CREATE/INSERT):

```
~/mariadb/mariadb-11.4.9-winx64/bin/mariadb.exe -h 127.0.0.1 -P 3307 -u iuasr_sis -p iuasr_sis < database.sql
```

Let op: dit **overschrijft** de bestaande gegevens in de database iuasr_sis. Maak eerst een verse back-up van de huidige stand als die nog waarde heeft.

Stap 5 — Configuratie controleren

- Controleer .env: DB_*-gegevens en APP_URL.
- **Laat APP_KEY ongewijzigd** (gelijk aan de back-up), anders zijn BSN/rekeningnummer niet te ontsleutelen.

Stap 6 — Cache legen & controleren

```
~/php/8.3/php.exe artisan optimize:clear
```

Een php artisan migrate is **niet** nodig: de dump bevat het volledige schema. Controleer tot slot of de applicatie start en of inloggen werkt.

5. Losse database-restore (zonder volledige recovery)

Alleen de gegevens terugzetten (code ongewijzigd)? Pak het archief uit (stap 1) en voer alleen stap 4 uit met `database.sql`.

6. AVG & beveiliging

- Back-ups bevatten alle persoonsgegevens én de encryptiesleutel — uitsluitend versleuteld en intern bewaren; niet e-mailen of naar buiten brengen.
- Echte productiedata alleen in de laatste fase, onder toezicht van de Functionaris Gegevensbescherming.
- Inzage/mutatie van cijfers en BSN wordt gelogd (audit-log, alleen-lezen voor Beheer).
- Rolscheiding wordt server-side afgedwongen; wijzig dit niet zonder reden.
- **Directie is opleidinggebonden.** De koppeltabel `directie_opleidingen` (user ↔ opleiding) bepaalt welke studenten, cijfers en rapporten een directielid ziet. Beheer wijst dit toe via **Gebruikers & rollen → Directie — opleidingtoewijzing**. Zonder toewijzing ziet een directielid **niets** (need-to-know). Een dubbel ingeschreven student is zichtbaar voor de directie van elke opleiding waarin hij/zij actief is. De filtering loopt via `User::opleidingIds()`, `Student::scopeZichtbaarVoor()` en per-opleiding gefilterde statistieken.
- **Presentiegegevens zijn onderwijsinhoudelijk.** Studentenzaken, Financiële Administratie en Beheer hebben géén toegang tot presentielijsten of aanwezigheidspercentages (Gate presentie-inzien). Registreren mag alleen de docent van het eigen vak (Gate presentie-registreren). Inzage en mutatie worden gelogd.

7. Presentie (aanwezigheidsregistratie)

De docent registreert per college de aanwezigheid; dit is verplicht. Het model is bewust **genormaliseerd**: één regel per student × vak × onderwijsweek — nooit vaste weekkolommen op de inschrijving.

Tabel	<code>presenties</code> (<code>inschrijving_id</code> , <code>vak_id</code> , <code>week</code> , <code>aanwezig</code> , <code>geregistreerd_door_id</code>) met unieke sleutel op (<code>inschrijving_id</code> , <code>vak_id</code> , <code>week</code>).
Regeling	Kolom <code>inschrijvingen.aanwezigheidsregeling_50</code> (boolean). Bewust op de inschrijving , niet op de student: zij geldt per opleiding en per studiejaar en moet bij herinschrijving opnieuw worden toegekend.
Normen	<code>config/sis.php</code> → <code>presentie.weken_per_blok</code> (8), <code>presentie.norm</code> (0.80) en <code>presentie.norm_regeling</code> (0.50).

Ontbrekende registratie is geen afwezigheid. Het percentage wordt berekend over de **geregistreerde** weken. Een week zonder regel telt niet mee — anders zou nalatigheid van de docent op de student worden afgewenteld. Een week geldt pas als “geregistreerd” wanneer alle presentieplichtige deelnemers een waarde hebben.

Vrijgestelde studenten (`vaktoewijzingen.vrijgesteld`) volgen het vak niet en worden bij het opslaan overgeslagen, óók als het formulier voor hen een waarde meestuurt. Wijzigt u `weken_per_blok`, dan blijven bestaande registraties met een hoger weeknummer in de database staan maar verdwijnen zij uit het scherm; ruim ze in dat geval expliciet op.

8. Regulier onderhoud

- **Migraties:** `php artisan migrate` na een update (reversible; maak eerst een back-up).
- **Cache:** `php artisan optimize:clear` bij onverwacht gedrag na wijzigingen.
- **Logs:** `storage/logs/` (zitten niet in de back-up).
- **Tests:** `php artisan test` moet groen zijn vóór uitrol.

Deze handleiding wordt bijgewerkt zodra er functies of infrastructuur wijzigen. Controleer de datum onderaan op actualiteit.